

PathFinder: Find Your Feel

In Greece, overtourism causes severe overcrowding in popular areas while hiding equally noteworthy trails, villages and gorges. To solve this, PathFinder deploys a multi-agent AI pipeline (Groq) that parses geospatial data (Waymarked Trails) and Eurostat regional tourism data to deliver sustainable itinerary recommendations for all hiker levels while actively prioritizing the reduction of overtourism.

Task Sequence

1. The Chatbot converses with the user to profile them based on the following characteristics:
 - a. hiker expertise / hiker intent
 - b. their noted interests

The chatbot also keeps track of the desired max and minimum user's preferred hike duration, as well as its location, either explicit or implicit

2. **WaymarkedTrails** provides the .json trail directory

For every waymark relation:

- We generated a clean sequential path array of coordinates from start to end, in order to define the *start and end coordinates* as json attributes for the trail.
- We used these coordinates and calculated the haversine (the direct surface distance in km between these coordinates). We added the attribute *distance_km*.
- We used the elevation profile to get the maximum elevation gain and loss, as attributes *elevation_gain_meters*, *elevation_loss_meters*
- Using the elevation gain/loss and total distance, we used a custom function to calculate the *difficulty_score* attribute:

$$\text{effort_distance} = \text{total_distance_km} + \left(\frac{\text{cumulative_gain}}{100.0} \right) + \left(\frac{\text{cumulative_loss}}{200.0} \right)$$

- Additionally, we use the Greek NUTS3 bounds to add the attribute *nuts3_code* to each trail

3. **Deterministic Step:** We **hard filter** the trail directory based on:
 - a. User's location. To handle the user's vague input on their preferred trail location we use the following spatial strategies.
 - i. If the user names a specific region (e.g. Attica), we use **Geolocator**, to limit trails to a bounding box returned by the API.
 - ii. If the user provides a time/distance limit (e.g., "30 minutes from here") we use the **Isochrone API**. Isochrone calculates a polygon representing areas reachable within a specific time frame. We use the generated box/polygon to filter out inaccessible trails.

- b. The hiker's intent ceiling. If the trail difficulty deviates more than 1 point from the hiker's perceived challenge demand, it is eliminated.
 - c. Duration.
 - i. If the user mentions "at most" or "at least" x hours, we can recommend only trails with that value as upper or lower limit.
 - ii. If the user mentions an approximate value, we allow a deviation of x times 10min, with a cap at an hour.
- All trails whose durations do not fit in the users recommended range or buffer are eliminated.

4. The Dynamic Multiplier Scoring Engine

- a. *Core metrics scale*: We look at the mathematical alignment of the numerical data. We score:
 - i. The difficulty of the path in a binary fashion: if it's equal to the user's perceived intended challenge level, it equals 1. Else, 0.
 - ii. The duration of the path, with decay scaling.
- b. *The Interests Score*: For each of the remaining trails, we use an agent for semantic evaluation, to match the description provided to the user's interests.
- c. *Anti-Tourism Score*: Overtourist trails are penalized with a static **Tourist Density Penalty** subtracted from the final score, calculated from **Eurostat** data. Trails in areas with high tourism density are less likely to be suggested. If there happens to be a high match score for the user, it is flagged as overcrowded.

$$\text{Final Match Score} = \text{Preference Alignment Score} - \text{Tourist Density Penalty}$$

With **CrewAI**, we deploy a 3 agent pipeline:

User Profiler Agent - Analyst Agent - Gatekeeper Agent

- The User Profiler Agent serves as the conversational interface with the user. It wraps structural data extraction inside a friendly dialogue persona. It tracks the 5 core pillars of a target hike (Intent, Expertise, Interests, Duration, and Location) and handles implicit semantic parsing (e.g., mapping "recovering from an injury" straight to low difficulty values).

The agent is assigned to the user_profiling() task to return the json attributes:

```
"hiker_expertise",
"hiker_intent",
"location_value",
"max_travel_time",
"travel_mode",
"spatial_strategy",
"min_trail_duration_hours",
"max_trail_duration_hours",
"interests"
```

- The Analyst Agent analyzes the chat history to gauge how uncompromising, rigid, or flexible a user is regarding the specifics of the core metrics scale. It evaluates behavioral tone to calculate relative importance weights and dynamically determines semantic value relevance for individual trail parameters.

The agent is assigned two tasks:

- Multiplier Extraction: The agent evaluates the global context of the entire chat history. It measures the user's psychological behavior—specifically tracking how flexible or strict they are about their constraints and in turn changing the matching scores.
 - Semantic Evaluation: The agent takes the user's processed interest keywords and maps them semantically against a raw trail description block, instead of using keyword matching.
- The Profile Gatekeeper Agent analyzes the user's latest incoming message against their active profile to catch structural context shifts (e.g., changing their mind about locations, modifying duration constraints, or requesting higher difficulties), without resetting the old user profile.

We set up the UI with **Streamlit**.

Assessment of Limitations

The web app could be further improved as follows:

- By including more data to map to user preferences, like:
 - Terrain type. Maybe a user mentions a past injury, the fact that they're carrying a stroller etc. Using available data on terrain types via ORS, we could include a terrain match score.
 - Weather conditions. Some users prefer sunny areas, others prefer gloomy and damp areas. Using the OpenWeatherMap API, we could give higher priority to trails with the desirable temperature, UV index, wind etc.
- By scouring the internet and using RAG verification:
 - We could read verified trail information regarding opening hours (and whether the trail is closed for the season), so as to not recommend to the user a closed trail.
 - By reading trail reports from News sources and online forums, an agent using RAG verification could assess safety concerns, as well as find a better match to the user's interests.